



Group 17 - Bomber Man

- Group Project Report

Submitted to: **Dr. Soumajit Pramanik**

Arpit Bhomia

12340340

arpitba@iitbhilai.ac.in

Arpit Kumar

12340350

arpitk@iitbhilai.ac.in

Ashish Ranjan

12340370

ashishr@iitbhilai.ac.in

Varun Rao

12342320

varunr@iitbhilai.ac.in

November 16, 2025

Bomber Man

Project Report

Abstract

This report presents a complete, classic Bomberman game developed using Python and the Tkinter library. The game challenges a player against multiple AI-controlled bots in a destructible, grid-based arena. Key features include a customizable game settings menu and an advanced AI system that uses different pathfinding algorithms and self-preservation tactics. The result is a dynamic and engaging arcade experience that balances classic gameplay with intelligent opponent behavior.

Contents

1	Problem	2
1.1	Game Definition	2
1.1.1	Rules	2
1.1.2	Game Controls	2
1.1.3	Features	3
2	Experiment	5
2.1	Approach	5
2.1.1	AI Bot Logic	5
2.2	Walkthrough	7
2.2.1	Initial Game State	7
2.2.2	Gameplay Phase	8
2.2.3	Demonstration Video	9
2.2.4	Github Repository of the source code	9
3	Libraries and Frameworks	10
3.1	Libraries and Frameworks	10
3.2	Dependencies	10
3.3	Run the Game	10
3.4	Core File Purposes	10
4	Results	11
4.1	Comparison between different pathfinding algorithms	11
4.2	Challenges and Solution	11
4.3	Future Improvement	12

1 Problem

1.1 Game Definition

This section introduces the core concept of the game, providing readers with a clear understanding of its purpose, mechanics, and target audience.

1.1.1 Rules

Objective:

The main goal is to be the last one standing. The player wins by defeating all AI-controlled bots on the map while maintaining at least one health point.

Core Gameplay Rules:

- **Player Health:** The player starts with 3 health points. The game is lost if health drops to zero.
- **Placing Bombs:** The player can place a limited number of bombs. The initial number of bombs and their explosion power can be adjusted in the game's settings.
- **Explosions:**
 - Bombs explode in a cross-shaped pattern extending in four directions (up, down, left, and right).
 - Explosions destroy soft walls (brownish blocks) in their path.
 - Explosions are blocked by hard walls (gray, indestructible blocks).
 - If an explosion hits another bomb, it triggers a chain reaction.
- **Taking Damage:** The player loses one health point when caught in any bomb blast, including their own bombs.
- **Enemies (Bots):**
 - Bots are AI-controlled enemies that move around and place bombs.
 - Each bot has 1 health point and is defeated by a single explosion.
 - Bots feature artificial intelligence that allows them to hunt the player when in proximity, destroy walls to clear paths, and attempt to escape from bomb blasts.

Win and Loss Conditions:

- **Victory:** Achieved by successfully defeating all bots on the map.
- **Defeat:** Occurs when the player's health is reduced to zero.

1.1.2 Game Controls

The game utilizes a keyboard-based control scheme for player interaction:

Movement:

- **Up:** Up Arrow key or W key
- **Down:** Down Arrow key or S key

- **Left:** Left Arrow key or A key
- **Right:** Right Arrow key or D key

Actions:

- **Place Bomb:** Spacebar

Exit Control:

- **Quit Game:** Q key or Escape key

1.1.3 Features

The game incorporates several key features designed to enhance user experience and strategic gameplay:

Core Gameplay Features:

- **Classic Bomberman Gameplay:** This is an implementation of the classic game where the player navigates a grid based arena, places bombs strategically, and battles enemies.
- **Player vs AI Mode:** The primary game mode features single player against multiple computer controlled bots.
- **Win/Loss Conditions:** Victory is achieved by eliminating all bots on the map. Defeat occurs when the player's health drops to zero.
- **Scoring System:** Players earn points for defeating enemies, with scores displayed on the bottom part of the display screen.

Map and Environment:

- **PCG Maps:** Each game map is dynamically generated with a strategic mix of indestructible hard walls and destructible soft walls, ensuring varied gameplay experiences.
- **Destructible Environment:** Players and bots can destroy soft walls using bombs to clear paths and locate opponents.
- **Bomb Mechanics:**
 - Bombs feature a set fuse time before detonation (2200ms).
 - Explosions propagate in a cross-shaped pattern (up, down, left, right).
 - Explosions can trigger chain reactions when multiple bombs are in proximity.

Advanced AI Opponents:

- **State-Driven AI:** Bots operate using a sophisticated three-state behavioral system:
 - *Search:* Bots actively seek and destroy soft walls to navigate the map and locate the player.
 - *Chase:* Bots pursue the player aggressively once the player enters their vision range.

- *Evade*: Bots immediately prioritize escape to safe locations when within the blast radius of active bombs.
- **Pathfinding**: Bots utilize the pathfinding algorithms(A*, Dijkstra, BFS) to calculate the most efficient routes to targets, whether pursuing the player or destroying walls.
- **Strategic Evasion**: The AI employs escape logic(modified Breadth First Search), searching for the safest and most open areas rather than simply the nearest safe tile, maximizing future movement options.
- **Self-Preservation Logic**: Bots will not place bombs unless they can calculate a viable escape route, preventing self entrapment and suicidal behavior.

Customization and User Interface:

- **Game Settings Screen**: A game menu allows players to customize their gameplay experience extensively.
- **Configurable Game Variables**: Players can adjust multiple parameters:
 - Number of bots in the game
 - Bot aggression level (probability of bomb placement)
 - Player's starting bomb count and explosion power
 - Overall game speed modifier
- **HUD Display**: The in-game interface provides real time information including player health, current score, available bomb count, and elapsed game time, and also Bot's movement and decision making.
- **Game Over Screen**: Victory (You Win!) or defeat (You Lose!) screens are displayed upon game conclusion.

2 Experiment

2.1 Approach

This section details the methodology and technical approach taken during the development process.

2.1.1 AI Bot Logic

The artificial intelligence system governing bot behavior is implemented through a sophisticated decision-making framework that balances aggression and exploration.

Core AI Loop (update_ai function):

The brain of the bot operates within the `update_ai` function, which is called periodically for each living bot.

1. **Thinking Logic:** To optimize performance and create more natural-feeling movement, bots do not make new decisions on every game tick. Decision-making occurs at fixed intervals (e.g., every 400ms via `think_interval_ms`). Between decision cycles, bots continue following their current path.
2. **State Determination:** At the beginning of each thought cycle, the bot evaluates its surroundings to determine its primary operational state. States are prioritized as follows:
 - *Evade*: Highest priority. Activated when the bot is standing on a tile within the blast radius of an imminent explosion.
 - *Chase*: Activated when the bot is not in immediate danger and the player is within its vision range (configurable Manhattan distance).
 - *Search*: Default state when the bot is not in danger and the player is not nearby. The bot roams the map seeking soft walls to destroy.

State 1: Evade

This represents the self-preservation state where survival is most important.

Trigger: The bot occupies a tile predicted to be within an explosion radius. The prediction logic (`predict_danger_times`) calculates affected tiles and remaining time until blast.

Action Sequence:

1. The bot invokes `find_safe_path`, (Modified Breadth First Search).
2. Rather than selecting any safe tile, the function performs a Breadth-First Search (BFS) to identify up to 5 nearest tiles outside any bomb's blast radius.
3. Each candidate safe tile is scored using `_calculate_openness`, which counts walkable neighbors. Since these tiles are more open the bot is less prone to get caught in the corner by the Player.
4. The final score combines openness and distance from the bot, with shorter paths receiving preference.
5. The bot calculates the path to the highest-scoring candidate tile.
6. This path is assigned to `bot.path`, and movement commences immediately.

Fallback: If `find_safe_path` fails to locate an escape route (e.g., trapped), the bot executes `random_move` in hope of reaching safety.

State 2: Chase

This represents the aggressive, player-hunting state.

Trigger: The player is detected within the bot's vision radius.

Action Sequence:

1. The bot's target is set to the player's current coordinates.
2. The pathfinding algorithms computes the shortest path to the player.
3. Path Stability: To prevent continuous path changes with minor player movements the bot maintains its current path for a duration (`BOT_REPATH_HOLD_MS`). New paths are calculated only when the current path is completed or blocked.
4. Bombing Logic: When proximity to the player reaches Manhattan distance of 2 or less, bomb placement is considered based on:
 - `bot.aggression`: A configurable probability value (0.1 to 1.0) determining likelihood of bomb placement.
 - `bot.can_escape_after_placing`: A critical safety check that simulates the explosion and verifies escape path existence. Bombs are not placed if escape is impossible, preventing suicide.
5. Upon bomb placement, `plan_escape_after_placing` immediately finds the quickest path to safety and transitions the bot to evade state.

State 3: Search

This represents the default exploratory state.

Trigger: The bot is not in danger and the player is beyond detection range.

Action Sequence:

1. The primary objective is destroying soft walls to clear the map and potentially locate the player.
2. The bot calls `find_nearest_soft` to identify the closest soft wall tile.
3. The pathfinding algorithms attempts to navigate toward the wall.
4. Upon approaching a soft wall, bomb placement may occur using the same aggression and safety checks as chase state.

Fallback: If no path to soft walls exists or the current path is blocked, `random_move` executes for map exploration.

Core Movement Logic

`follow_path_step` function: Executes sequential movement along a calculated path.

1. Retrieves the next coordinate from the path.
2. Safety Verification: Performs critical danger assessment before movement. If the next step leads to a dangerous tile, the entire path is discarded (`bot.path = []`) and a stale flag is set, forcing situation re-evaluation in the next thought cycle.
3. Verifies the tile is walkable and unoccupied by other entities.

4. If all checks pass, bot coordinates are updated and the step is removed from the path.
5. If unexpected obstacles block the path, it is cleared to force recalculation.

`random_move` function: Implements basic exploratory movement by evaluating four neighboring tiles, randomly shuffling them, and moving to the first safe and walkable option.

2.2 Walkthrough

2.2.1 Initial Game State

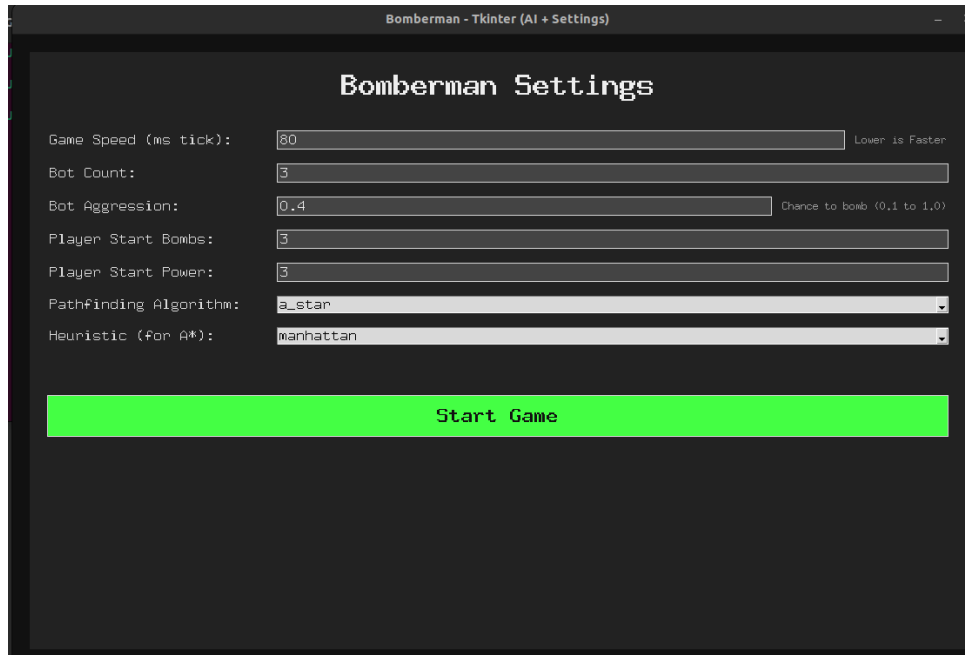


Figure 1: Initial game screen showing the starting options.

Available Options:

- **Game Speed:** Modify the overall pace of gameplay.
- **Number of Bots:** Adjust the quantity of AI opponents (range: 1-10 bots)
- **Bot Aggression:** Configure the likelihood of bots placing bombs (range: 0.1-1.0)
- **Player Bomb Count:** Define the starting number of bombs available to the player
- **Player Bomb Power:** Set the initial explosion radius of player bombs
- **Start Game:** Begin gameplay with the configured settings
- **Pathfinding Algorithm:** Set the algorithm for AI bot (A*, Dijkstra, or BFS)
- **Heuristic:** Select the heuristic in case of A* Algorithm.

2.2.2 Gameplay Phase

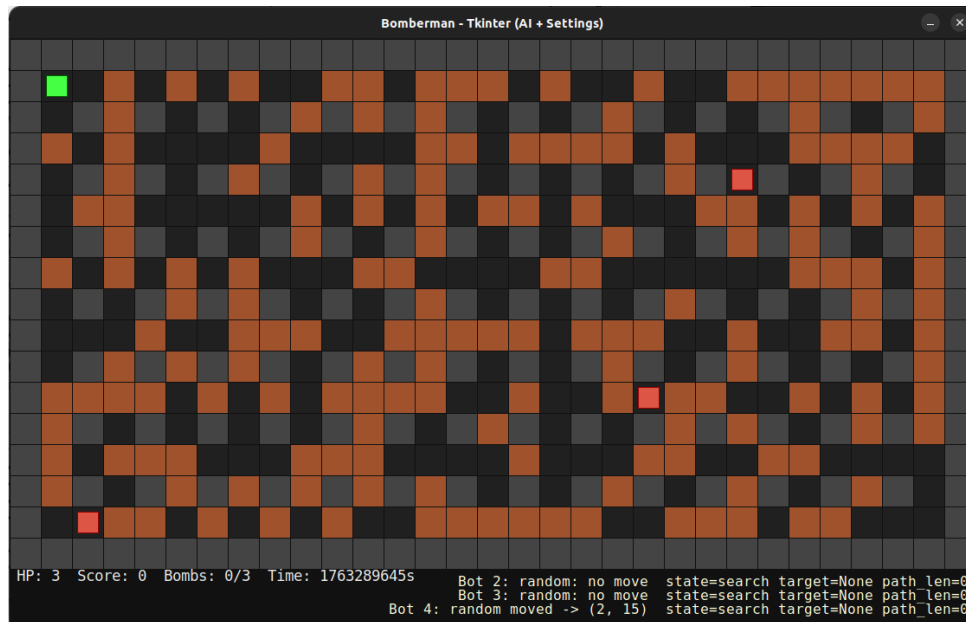


Figure 2: Active gameplay showing main game mechanics in action.

The game features a grid-based map with various elements:

- **Player Character (Green Square):** Controllable player, aiming to eliminate bots.
- **Bot Characters (Red Squares):** AI enemies that move, place bombs, and hunt the player.
- **Hard Walls (Dark Grey Blocks):** Indestructible barriers that define the map layout.
- **Soft Walls (Brownish Blocks):** Destructible blocks that can be cleared by bombs.
- **Empty Tiles (Dark Background):** Walkable areas where movement and bomb placement occur.
- **Bombs (Yellow Circles):** Player/bot-placed explosives with a fuse, causing explosions.
- **Explosions (Orange Blasts):** Temporary damage zones from bomb detonations, harming entities and destroying soft walls.

Heads-Up Display (HUD):

- **HP:** current health.
- **Score:** accumulated points.
- **Bombs:** Active bombs vs. maximum allowed.
- **Time:** Elapsed game time.
- **Bot ID:** Unique identifier.
- **Status:** Current action/intention.
- **State:** AI's current objective (search, chase, evade).
- **Target:** Destination coordinates.

2.2.3 Demonstration Video

A comprehensive video demonstration of the game showcasing all features, gameplay mechanics, and AI behavior is available here.



Figure 3: Click on the image to watch the full gameplay demonstration video.

Watch Online: <https://www.youtube.com/watch?v=fGdCZeGzFvk>

2.2.4 Github Repository of the source code

Code - [click here](#)

3 Libraries and Frameworks

3.1 Libraries and Frameworks

- **Tkinter:** The game is built using Tkinter, which is Python's standard, built-in library for creating graphical user interfaces (GUIs). It is used to draw the game window, settings screen, map, characters, and all other visual elements.

3.2 Dependencies

- **Python 3:** The only dependency required to run this game is a standard installation of Python (version 3.x).

3.3 Run the Game

- Navigate to the project folder in your terminal.
- Run the following command:

```
python3 main.py
```

This will launch the "Bomberman Settings" window, and from there you can start the game.

3.4 Core File Purposes

- **main.py:** The main entry point that starts the application and manages switching between the settings and game screens.
- **game.py:** Contains the core game logic, including the main loop, player input, AI behavior, and rendering everything to the screen.
- **settings.py:** Defines the pre-game settings screen where the player can configure options like bot count and aggression.
- **config.py:** Stores all global constants and default settings, such as map dimensions, bomb fuse time, and health points.
- **entities.py:** Defines the data structures for all game objects, including the Player, Bots, Bombs, and Explosions.
- **map.py:** Responsible for generating the grid-based game map with its pattern of hard and soft walls.
- **pathfinding.py:** Implements different pathfinding algorithms (A*, Dijkstra, BFS) that allows the bots to navigate the map intelligently.
- **utils.py:** A collection of small, reusable helper functions used across the project.

4 Results

4.1 Comparison between different pathfinding algorithms

Algorithm	Mechanism	Nodes Expanded (Stats)	Analysis & In-Game Behavior
A*	A "best-first" search algorithm that uses an admissible heuristic (Manhattan distance) to guide its path, prioritizing tiles that are closer to the target.	approx. 28	Excellent: This is the optimal choice. The heuristic makes the search incredibly efficient, resulting in fast calculations and highly responsive bots that move decisively.
Dijkstra's	An uninformed search algorithm that explores outwards from the start node in ever-expanding circles of distance, without any knowledge of the goal's direction.	approx. 85	Poor: By checking every possible path in order of length, it wastes significant processing time on irrelevant areas. Bots will feel less responsive, and game performance may degrade with many bots on screen.
BFS	A simple uninformed search that explores all neighbors at the current depth before moving to the next. On a grid, its exploratory pattern is functionally identical to Dijkstra's.	approx. 85	Poor: Like Dijkstra's, it is inefficient for finding a path to a specific target. It checks over 3 times as many nodes as A*, leading to the same performance issues.

Table 1: Statistical comparison of selectable pathfinding algorithms.

4.2 Challenges and Solution

The development of this Bomberman game presented several significant technical and design challenges that required careful analysis and creative problem-solving approaches.

- **Bot Aggression: Smart vs. Reckless**

- **Problem:** A highly aggressive bot would often kill itself by placing bombs without thinking, making it a weak opponent.
- **Solution:** A bot's aggression is now its desire to attack, but it's always overruled by a mandatory safety check. Before placing a bomb, the bot first confirms it has a safe escape route. This makes aggressive bots both dangerous and smart.

- **AI Movement: Decisive vs. Indecisive**

- **Problem:** The AI would constantly change its path while chasing a moving player, causing it to flicker in place and hardly move.
- **Solution:** The bot now only makes a new decision every few moments, not every single frame. This forces it to stick to its chosen path for a short duration, making its movement appear more deliberate and effective.

- **Bot Speed: Balanced vs. Unfair**

- **Problem:** At high game speeds, bots could move with superhuman speed, making the game impossible for the player.

- **Solution:** The bot's movement speed is now balanced against the game's speed. Even if the game runs faster, the bot's speed remains proportional to the player's, ensuring the game stays fair and challenging.
- **Entity Collision: Physical vs. Ghost-like**
 - **Problem:** Bots and players could walk through each other, which breaks the game's logic and strategy.
 - **Solution:** An occupancy check was added. Before moving, each character checks if the target tile is already taken by another character. This gives them physical presence, allowing for strategies like trapping and cornering.
- **Bombing Strategy: Tactical vs. Suicidal**
 - **Problem:** Bots would place bombs to attack the player but would often get caught in their own blast.
 - **Solution:** The AI now follows a two-step tactical loop. First, it simulates to ensure an escape route exists before placing a bomb. Second, after placing it, the bot's immediate priority becomes following that escape path to safety.
- **Overall Feel: Realistic vs. All-Knowing AI**
 - **Problem:** A perfect AI that reacts instantly to everything feels like it's cheating and is not fun to play against.
 - **Solution:** The bot is intentionally designed to not be all-knowing. It only assesses its surroundings periodically, creating a gap. This gives the player a window of opportunity to lay a trap that the bot might not notice until it's too late, making the game feel fair and rewarding.
- **Game Architecture: (C++ to Python)**
 - **Problem:** A C++ terminal-based version would naturally become turn-based, where the program waits for player input before letting the bot move. This synchronous loop loses the real-time urgency, and there is no easy way to handle concurrent events like bomb timers and independent AI thinking except making another window for output and using custom animation and sprite textures.
 - **Solution:** Using the Python Tkinter library overcomes this. It provides not only an easy-to-use graphical interface but also a global event timer (`root.after()`) that enables concurrency, allowing player movement, bot AI, and bomb fuses to all run simultaneously and independently.

4.3 Future Improvement

- **Prioritize Smart Wall Breaking:** The AI could be enhanced to analyze the map and strategically destroy walls that open up the largest areas or create the most direct path to the player. This would make the bot's exploration feel more purposeful rather than just targeting the nearest block.
- **Power-Ups:** Items like Speed Up or Extra Bomb could be hidden within soft walls, creating a new objective for both the player and the AI. Bots would need to be programmed to recognize, evaluate the strategic value of, and safely collect these power-ups.
- **Baiting and Trapping:** A more advanced AI could learn to trap the player by placing bombs to cut off escape routes rather than just attacking directly. This would involve the bot predicting the player's likely movement and using the environment against them.

- **AI Working as a Team:** Bots could share information, such as the player's last known location or their own current action (e.g., chasing, escaping). This would enable coordinated tactics like pincer movements, where one bot flushes the player towards another.
- **Specific Bot Roles:** Instead of being identical, bots could be assigned roles like Chaser, Trapper, or Demolitionist. A Chaser's priority would be to pursue the player, while a Trapper focuses on cornering them and a Demolitionist on clearing the map.
- **Audio and Sprite Enhancements:** The game's presentation could be greatly improved by replacing the basic shapes with animated 2D sprites for characters and explosions. Adding sound effects for actions like walking, placing bombs, and explosions would provide crucial audio feedback and make the game more immersive.
- **Chain Reaction Management:** The AI could be upgraded to understand the physics of chain reactions, allowing it to use them both offensively and defensively. Offensively, it could place a bomb to intentionally set off a sequence of other bombs to trap a player; defensively, it could more accurately predict the full, cascading blast radius to find a truly safe escape path.

Conclusion

In conclusion, this project successfully delivered a complete and engaging Bomberman game using Python and Tkinter. The primary achievement was the development of a sophisticated AI that uses pathfinding, state management, and self-preservation to create a challenging and fair opponent. The final product effectively captures the strategic essence of the classic arcade game while providing modern customization options, demonstrating a successful fusion of retro gameplay with intelligent software design.